

Desarrollo de Software en Sistemas Distribuidos

Cátedra: Azcurra – Amaro - Tomczak.

Cuatrimestre: Segundo

Año: 2025

TRABAJO PRÁCTICO

ONG-EMPUJE-COMUNITARIO

Grupo E: Integrantes

Nombre	<i>Federico Acosta</i>	<i>Nahuel Rivero</i>	<i>Gastón Romero</i>	
Tareas Realizadas	- Frontend - Vistas - Formularios-JWT,roles y autenticación de usuarios. - Eventos kafka	- Cliente - Lógica negocio - Protos cliente - Transferir y Ofrecer donaciones kafka - Informe excel de donaciones(RE ST) - Consulta de presidentes y ONGs(SOAP)	- Servidor grpc - Servicios grpc - Protos grpc - Tests grpc - Solicitud de Donaciones Kafka - Cancelar Solicitud de Donaciones Kafka	
DNI	38.839.452	40.848.267	45.873.095	

Repositorio Github: <https://github.com/FedeOA/ong-empuje-comunitario>

Primera Parte

Pendientes: *Quedó pendiente conectar el frontend con el cliente y con el servidor rpc para el módulo de gestión de eventos.*

Estrategia de resolución

Tecnologías utilizadas: *Para el proyecto ong-comunitario se han empleado 3 vertientes:*

Frontend:

*El frontend fue desarrollado utilizando **React 19.1.1** con **React Router DOM** para el manejo de rutas y **jwt-decode** para la gestión de autenticación mediante tokens JWT. Para el entorno de desarrollo y compilación se empleó **Vite 7.1.2** junto con **@vitejs/plugin-react-swc**, lo que permitió tiempos de compilación muy rápidos y soporte para Hot Module Replacement.*

*Para el diseño y estilos, se utilizó **Tailwind CSS**, optimizado mediante **PostCSS** y **Autoprefixer** para garantizar compatibilidad entre navegadores.*

Cliente:

Se ha utilizado el lenguaje de programación Java con la versión 24 y Spring Boot con la versión 3.5.5.

Servidores:

Se ha utilizado el lenguaje Python con la versión 3.13.7 y pip versión 25.2.

Base de datos:

Se ha utilizado MySQL versión 9.4.0 y MySQL Server la versión 9.4.0.

Descripción de la resolución: *El procedimiento de resolución ha sido el siguiente, se ha*

optado por la implementación de un sistema RPC siguiendo el modelo de 4 capas, el cuál consistió en:

Frontend:

Desarrollar el frontend utilizando un enfoque basado en componentes reutilizables y un manejo eficiente del estado (con hooks y/o contextos) para ofrecer una experiencia de navegación fluida e interactiva. Se implementará enrutamiento dinámico para la gestión de páginas, actualización reactiva del DOM virtual para optimizar el rendimiento, y buenas prácticas de accesibilidad y diseño responsivo para garantizar una experiencia de usuario consistente en distintos dispositivos.

Se estará ejecutando en el puerto "5173", y posee las características:

- */Login Login de usuarios con vista pública.*
- *Vista de /Home de acuerdo al rol del usuario. Las vistas default obtienen los datos y los listan en una tabla.*
- *Vistas/modales para agregar/dar de baja/ modificar usuarios*
- *Vistas/modales para agregar/dar de baja/ modificar donaciones*
- *Vistas/modales para agregar/dar de baja/ modificar eventos*
- *Perfil de usuario: Avatares de acuerdo a rol, nombre de usuario y rol. Botón para cerrar sesión.*
- *Manejo de token en local storage para almacenar el rol y el nombre de usuario*
- *Cerrado de sesión*

Cliente: *Se prosiguió con el desarrollo del cliente en Java para dar soporte a los endpoints que el frontend solicitará, comunicándose a su vez con los servidores para realizar las queries pertinentes, los cuáles estarán a la escucha en el puerto 8080, gestionando la comunicación entre las vistas y los servicios. Para organizar el proyecto se definieron los siguientes packages:*

config: se encarga de la configuración de Spring Boot y gRPC, incluyendo la inicialización de canales, beans de conexión, y propiedades globales de la aplicación.

controller: Contiene los controladores REST implementados con spring boot, los cuales exponen los endpoints que consumirá el frontend y traduce la petición HTTP en una invocación gRPC hacia el servicio correspondiente.

service: Aquí se encuentra la lógica de negocio. Se realizan las llamadas a los stubs gRPC generados a partir de los archivos .proto, se manejan las respuestas y se traducen de manera que entiende el frontend.

dto: Contiene las clases DTO para mapear de manera clara la información que viaja entre el gateway y el frontend.

exception: Contiene el manejo de errores, devuelve mensajes al frontend al fallar la comunicación o alguno de los servicios.

mapper: Define la lógica para convertir entidades de modelos gRPC en DTOs que el frontend pueda interpretar, y viceversa.

Servidores: *Se continuó con el desarrollo de los servidores que se encargará de contactar al cliente para devolver los resultados de las queries realizadas a la base de datos, y también de ejecutar las instrucciones HQL para contactar la base de datos, el cual estará escuchando en el puerto 50051, el cuál consistió en separar los sub-módulos dentro del módulo servers de la siguiente manera:*

Sub-módulo database: Que contendría las clases correspondientes para armar los requests a la base de datos y para enviar al cliente, contenía también la configuración para comunicarse y entablar conexión con la base de datos, a su vez para establecer una sesión con la misma.

Sub-módulo services: El cuál contendría toda la lógica de instrucciones SQL para consultar la base de datos, se ha desarrollado el UserService.py, el EventService.py, el DonationService.py y el AuthService.py. Esto incluye tanto request de login, consulta de inventario, miembros adheridos a eventos y roles de usuarios.

Sub-módulo services pb2: Se encarga de definir las firmas/interfaces necesarias para ejecutar los servicios GRPC definidos en los services pb2 grpc, los cuales contendrán la definición de los métodos correspondientes, son generados por Protocol Buffers.

Sub-módulo services pb2 grpc: Son las definiciones de los métodos definidos en las interfaces de los services pb2, el cuál se encarga principalmente de la comunicación del servidor con el cliente.

Sub-módulo test: Contiene la definición de todos los métodos correspondientes definidos en los services, los cuáles son testeados automáticamente verificando integridad de la consulta SQL como también de los resultados obtenidos de la ejecución de dicha funcionalidad, se ha desarrollado el test para TestUser.py, TestEvent.py, TestDonation.py, TestAuthentication.py, con sus respectivos métodos.

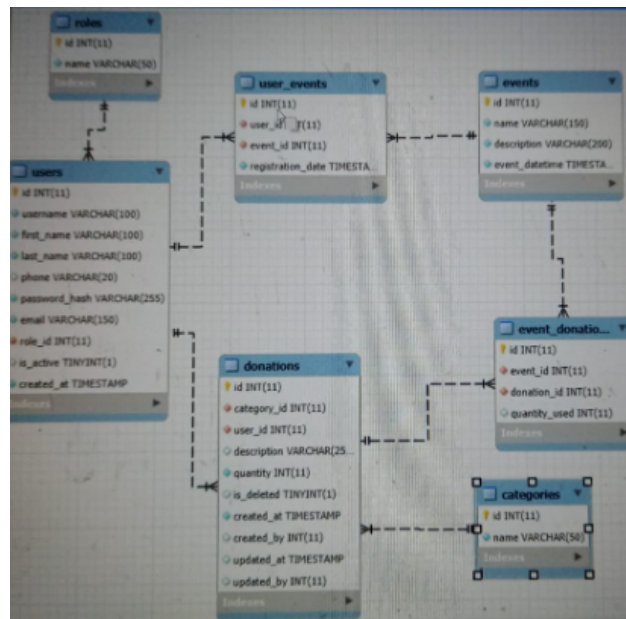
Finalmente se desarrolló una base de datos relacional MySQL, la cuál ha sido desarrollada e implementada con MySQL Server y MySQL Workbench, se ha utilizado localhost para alojar la información de la base de datos. El grado de normalización aplicado es quinta forma normal, las tablas correspondientes han sido Role, Users, Events, Donations, Categories, User_Event, Event_Donation.

Diagramas: *Los diagramas utilizados han sido los siguientes:*

Arquitectura utilizada:



Diagrama de base de datos utilizado:



Problemas/Errores: Los problemas detectados durante el desarrollo han sido los siguientes:

Frontend: El principal desafío fué la integración con el cliente rcp, teniendo en cuenta que se debieron utilizar las mismas interfaces o contratos para la comunicación entre ambos, y al mismo tiempo esto debía ser coincidente con los contratos soportados por el servidor rcp.

Cliente: El principal problema fue en relación a las dependencias ya que existían conflictos entre algunas de ellas por lo que se tuvo que eliminar la dependencia conflictiva.

Problemas en relación a la compatibilidad entre las versiones de gRPC y el starter Spring boot

Servidores: Los errores detectados en los servidores han tenido relevancia principalmente con la definición de los services, la incorrecta definición de los métodos en función de las queries a la base de datos. El nombre de las tablas ejecutadas ha sido erróneo, ocasionando errores de conexión y establecimiento de consultas en la base de datos.

Inconsistencia entre los archivos proto grpc y los services, errores de ejecución producto de no haber definido los stub correspondientes o los service servicer en los grpc files, generados mediante el protocol buffer, ocasionó errores de ejecución.

Error de compatibilidad con determinadas versiones de pymysql o de librerías como sqlalchemy, en dónde la versión de python actual del sistema resultaba incompatible, se resolvió creando un entorno virtual e instalando una versión compatible y estable de python.

Base de datos: Principalmente han sido errores de integridad referencial, se han detectado inconsistencias de referencias nulas en claves foráneas en casos de inserción de datos mediante Creates por medio de los services de los servidores grpc.

Errores de lógica al definir incorrectamente el nombre de las tablas de la base de datos, lo que ocasionó conflictos con las queries en los servidores, lo mismo sucedía con las columnas de las tablas.

Inconvenientes generados a la hora de hacer consultas a ids referencias por claves foráneas que resultaban inexistentes en la base de datos, asignación de referencia nula cuando en los servidores aún se conservaba en memoria la referencia hacia dichos ids.

Pruebas

Frontend:

/Login



Iniciar Sesión

Correo electrónico o usuario

fedecosta07@hotmail.com

Contraseña

.....

Ingresar

/home

Presidente con vista default en gestión de usuarios



President
PRESIDENTE

Gestión de Usuarios

Gestión de Eventos

Donaciones

Usuarios

Agregar Usuario

Usuario	Nombre	Apellido	Teléfono	Email	Rol	Activo	Acciones	
President	president	president	123456789	fedecosta07@hotmail.com	PRESIDENTE	Si	Modificar	Baja
Juan123	Juan	Ramirez	12312312	Juan@hotmail.com	VOLUNTARIO	Si	Modificar	Baja
MatiasYSH	Matias	Bueno	12312312	mati@hotmail.com	VOLUNTARIO	Si	Modificar	Baja
DiegoYsh	Diego	Juarez	12312312	diegoH2o@hotmail.com	VOLUNTARIO	Si	Modificar	Baja



Gestión de baja/alta lógica de los usuarios



President
PRESIDENTE

Gestión de Usuarios

Gestión de Eventos

Usuarios

Agregar Usuario

Usuario	Nombre	Apellido	Teléfono	Email	Rol	Activo	Acciones
President	president	president	123456789	fedecosta07@hotmail.com	PRESIDENTE	Si	<button>Modificar</button> <button>Baja</button>
Juan123	Juan	Ramirez	12312312	Juan@hotmail.com	VOLUNTARIO	No	<button>Modificar</button> <button>Alta</button>

Gestión de alta y modificación de usuarios



President
PRESIDENTE

Gestión de Usuarios

Gestión de Eventos

Donaciones

Usuarios

Agregar Usuario

Usuario	Nombre	Apellido	Teléfono	Email	Rol	Activo	Acciones
President	president	president	123456789	fedecosta07@hotmail.com	PRESIDENTE	Si	<button>Modificar</button> <button>Baja</button>
Juan123	Juan	Ramirez	12312312	Juan@hotmail.com	VOLUNTARIO	Si	<button>Modificar</button> <button>Baja</button>
MatiasYSH	Matias	YSH			VOLUNTARIO	Si	<button>Modificar</button> <button>Baja</button>
DiegoYsh	Diego	Ysh			VOLUNTARIO	Si	<button>Modificar</button> <button>Baja</button>
DiegoYsh	Diego	Ysh			VOCAL	Si	<button>Modificar</button> <button>Baja</button>
username	username				VOCAL	Si	<button>Modificar</button> <button>Baja</button>
username	username				VOCAL	Si	<button>Modificar</button> <button>Baja</button>
presidente	presidente				PRESIDENTE	Si	<button>Modificar</button> <button>Baja</button>
usuario2	sadada	Martinez	1189412312	fasda@hotmail.com	VOLUNTARIO	Si	<button>Modificar</button> <button>Baja</button>

Agregar Usuario

Usuario

Nombre

Apellido

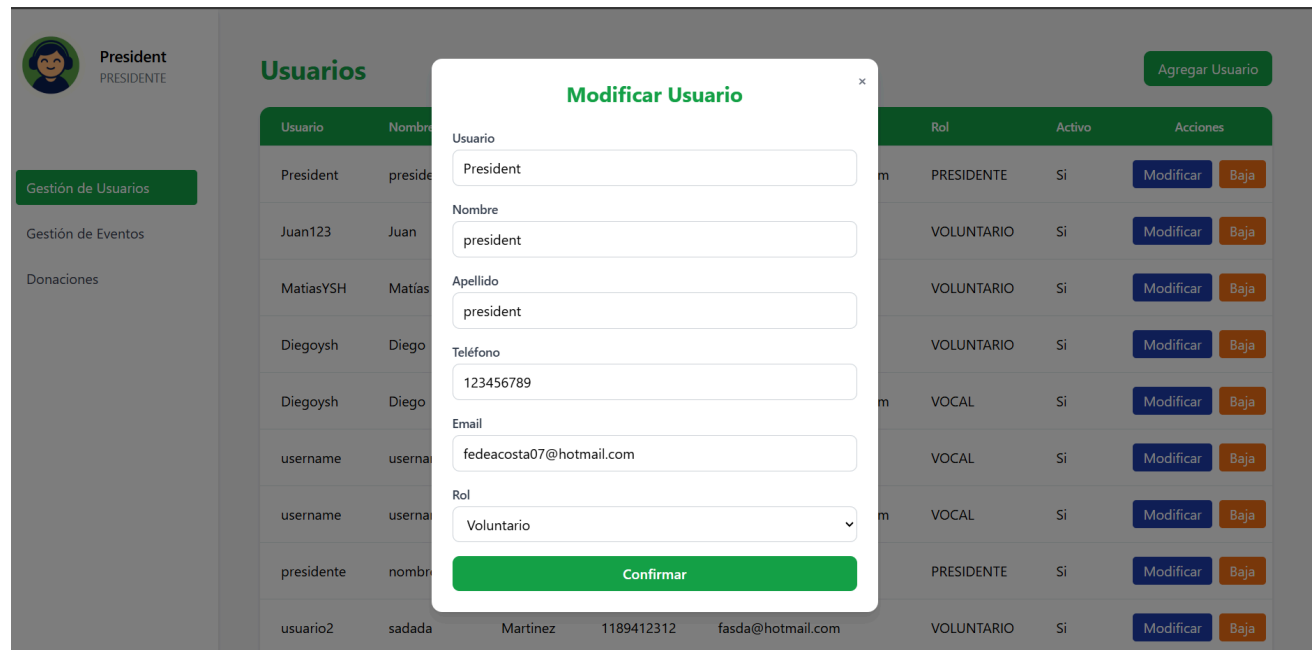
Teléfono

Email

Rol

Voluntario

Agregar Usuario



Vocal



President
VOCAL

Donaciones

Donaciones				Registrar Donación
Descripción	Cantidad	Categoría	Acciones	
Camisa	3434	ROPA	<button>Modificar</button>	
Pantalón	3	ROPA	<button>Modificar</button>	

Gestión de alta y modificación de donaciones





President
VOCAL

Donaciones

Donaciones

Registrar Donación

Descripción	Cantidad	Categoría	Acciones
Camisa			Modificar
Pantalon			Modificar

Registrar Donación

Descripción

Descripción de la donación

Cantidad

Cantidad

Categoría

Categoría de la donación

Agregar Donación



President
VOCAL

Donaciones

Donaciones

Registrar Donación

Descripción	Cantidad	Categoría	Acciones
Camisa			Modificar
Pantalon			Modificar

Modificar Donación

Descripción

Camisa

Cantidad

Cantidad

Categoría

ROPA

Confirmar

/home

Voluntario con vista default en gestión de eventos(falta implementar que solo se pueda agregar al evento)



President
VOLUNTARIO

Gestión de Eventos

Eventos Solidarios

Agregar Evento

Nombre	Descripción	Fecha	Miembros	Acciones
Campaña de alimentos	Recolección y distribución de alimentos.	2025-10-15	Ver miembros	Agregarse Modificar Eliminar
Limpieza del barrio	Jornada de limpieza comunitaria.	2024-12-01	Ver miembros	Agregarse

/home

Coordinador con vista default en gestión de eventos.



President
COORDINADOR

Gestión de Eventos

Eventos Solidarios

Agregar Evento

Nombre	Descripción	Fecha	Miembros	Acciones
Campaña de alimentos	Recolección y distribución de alimentos.	2025-10-15	Ver miembros	Agregarse Modificar Eliminar
Limpieza del barrio	Jornada de limpieza comunitaria.	2024-12-01	Ver miembros	Agregarse

Cliente:

Servidores:

```
(venv) PS C:\Users\gasto\Documents\GitHub\ong-empuje-comunitario\servers> python main.py
Compilando archivos .proto...
Compilación exitosa
Corrigiendo importaciones en archivos gRPC y Protobuf...
Corregido import en C:\Users\gasto\Documents\GitHub\ong-empuje-comunitario\servers\services_pb2_grpc\user_pb2_grpc.py
Corregido import en C:\Users\gasto\Documents\GitHub\ong-empuje-comunitario\servers\services_pb2\user_pb2.py
Corregido import en C:\Users\gasto\Documents\GitHub\ong-empuje-comunitario\servers\services_pb2_grpc\authorize_pb2_grpc.py
Corregido import en C:\Users\gasto\Documents\GitHub\ong-empuje-comunitario\servers\services_pb2\authorize_pb2.py
Corregido import en C:\Users\gasto\Documents\GitHub\ong-empuje-comunitario\servers\services_pb2_grpc\event_pb2_grpc.py
Corregido import en C:\Users\gasto\Documents\GitHub\ong-empuje-comunitario\servers\services_pb2\event_pb2.py
Corregido import en C:\Users\gasto\Documents\GitHub\ong-empuje-comunitario\servers\services_pb2_grpc\donation_pb2_grpc.py
Corregido import en C:\Users\gasto\Documents\GitHub\ong-empuje-comunitario\servers\services_pb2\donation_pb2.py
2025-09-17 14:34:40,756 INFO sqlalchemy.engine.Engine SELECT DATABASE()
2025-09-17 14:34:40,757 INFO sqlalchemy.engine.Engine [raw sql] {}
2025-09-17 14:34:40,759 INFO sqlalchemy.engine.Engine SELECT @@sql_mode
2025-09-17 14:34:40,759 INFO sqlalchemy.engine.Engine [raw sql] {}
2025-09-17 14:34:40,761 INFO sqlalchemy.engine.Engine SELECT @@lower_case_table_names
2025-09-17 14:34:40,761 INFO sqlalchemy.engine.Engine [raw sql] {}
2025-09-17 14:34:40,763 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2025-09-17 14:34:40,764 INFO sqlalchemy.engine.Engine DESCRIBE `ong-empuje-comunitario`.`users`
2025-09-17 14:34:40,764 INFO sqlalchemy.engine.Engine [raw sql] {}
2025-09-17 14:34:40,772 INFO sqlalchemy.engine.Engine DESCRIBE `ong-empuje-comunitario`.`donations`
2025-09-17 14:34:40,773 INFO sqlalchemy.engine.Engine [raw sql] {}
2025-09-17 14:34:40,777 INFO sqlalchemy.engine.Engine DESCRIBE `ong-empuje-comunitario`.`events`
2025-09-17 14:34:40,782 INFO sqlalchemy.engine.Engine [raw sql] {}
2025-09-17 14:34:40,786 INFO sqlalchemy.engine.Engine DESCRIBE `ong-empuje-comunitario`.`roles`
2025-09-17 14:34:40,807 INFO sqlalchemy.engine.Engine [raw sql] {}
2025-09-17 14:34:40,812 INFO sqlalchemy.engine.Engine DESCRIBE `ong-empuje-comunitario`.`event_donations`
2025-09-17 14:34:40,813 INFO sqlalchemy.engine.Engine [raw sql] {}
2025-09-17 14:34:40,807 INFO sqlalchemy.engine.Engine [raw sql] {}
2025-09-17 14:34:40,812 INFO sqlalchemy.engine.Engine DESCRIBE `ong-empuje-comunitario`.`event_donations`
2025-09-17 14:34:40,813 INFO sqlalchemy.engine.Engine [raw sql] {}
2025-09-17 14:34:40,833 INFO sqlalchemy.engine.Engine COMMIT
Servidor gRPC iniciado en puerto 50051
```

Ac

Se ejecutó el archivo que actúa de servidor central, el cuál aceptará y gestionará todas las peticiones desde el lado del servidor atendiendo el puerto 50051.

```
(venv) PS C:\Users\gasto\Documents\GitHub\ong-empuje-comunitario\servers\test> python TestUser.py
Starting UserService Tests
=====

=== Testing UserService ===
CreateUser: success=True, message=User created successfully and password emailed
ListUsers: found 9 users
- User: id=1, username=Leonel, email=updated@example.com, last_name=Doe, role_id=1
- User: id=2, username=Janedoe, email=jane.doe@example.com, last_name=Doe, role_id=2
- User: id=3, username=Bobsmith, email=bob.smith@example.com, last_name=Smith, role_id=2
- User: id=7, username=Sam, email=eventuser@example.com, last_name=Doe, role_id=1
- User: id=8, username=Ben, email=updated@example.com, last_name=Smith, role_id=3
om, last_name=Jhonson, role_id=1
- User: id=22, username=testuser_20250917143553, email=gastonromero210@example.com, last_name=User, role_id=1
UpdateUser: success=True, message=User updated successfully
om, last_name=Jhonson, role_id=1
- User: id=22, username=testuser_20250917143553, email=gastonromero210@example.com, last_name=User, role_id=1
UpdateUser: success=True, message=User updated successfully
DeleteUser: success=True, message=User deleted successfully

=====
UserService Tests Completed
(venv) PS C:\Users\gasto\Documents\GitHub\ong-empuje-comunitario\servers\test>
```

Se ejecutó el archivo TestUser, el cuál tiene las definiciones para la evaluación de los métodos CreateUser, UpdateUser, DeleteUser, ListUsers.



```
(venv) PS C:\Users\gasto\Documents\GitHub\ong-empuje-comunitario\servers\test> python TestEvent.py
Starting EventService Tests
=====

=== Testing EventService ===
CreateEvent: success=True, message=Event created successfully
ListEvents: found 6 events
- Event: id=1, name=Community Cleanup, fecha_hora=2025-10-01T10:00:00
- Event: id=2, name=Food Drive, fecha_hora=2025-10-15T14:00:00
- Event: id=7, name=Parish visit, fecha_hora=2025-09-13T19:57:48
- Event: id=8, name=Visit orphanage, fecha_hora=2025-09-13T20:01:32
- Event: id=11, name=Distribution of the garment, fecha_hora=2025-09-15T15:42:56
- Event: id=16, name=Test Event, fecha_hora=2025-09-17T14:36:59
AddUser: success=True, message=User added to event
- Users in event 16: ['eventuser_20250917143654']
RemoveUser: success=True, message=User removed from event
- Users in event 16: []
UseDonations: success=True, message=Donation used in event
2025-09-17 14:36:58,951 INFO sqlalchemy.engine.Engine SELECT DATABASE()
2025-09-17 14:36:58,951 INFO sqlalchemy.engine.Engine [raw sql] {}
2025-09-17 14:36:58,955 INFO sqlalchemy.engine.Engine SELECT @@sql_mode
2025-09-17 14:36:58,955 INFO sqlalchemy.engine.Engine [raw sql] {}
2025-09-17 14:36:58,956 INFO sqlalchemy.engine.Engine SELECT @@lower_case_table_names
2025-09-17 14:36:58,956 INFO sqlalchemy.engine.Engine [raw sql] {}
2025-09-17 14:36:58,957 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2025-09-17 14:36:58,961 INFO sqlalchemy.engine.Engine SELECT event_donations.id AS event_donations_id, event_donations.quantity_used AS event_donations_quantity_used, event_donations.event_id AS even
t_donations_event_id, event_donations.donation_id AS event_donations_donation_id
FROM event_donations
WHERE event_donations.event_id = %(event_id_1)s AND event_donations.donation_id = %(donation_id_1)s
LIMIT %(param_1)s
2025-09-17 14:36:58,961 INFO sqlalchemy.engine.Engine [generated in 0.00044s] {'event_id_1': 16, 'donation_id_1': 23, 'param_1': 1}
- EventDonation created: event_id=16, donation_id=23, quantity_used=1
2025-09-17 14:36:58,965 INFO sqlalchemy.engine.Engine ROLLBACK
UpdateEvent: success=True, message=Event updated successfully
DeleteEvent: success=True, message=Event deleted successfully

=====
EventService Tests Completed
(venv) PS C:\Users\gasto\Documents\GitHub\ong-empuje-comunitario\servers\test>
```

Activar Windows

Ejecución del archivo TestEvent, el cuál contiene las definiciones de los métodos CreateEvent, UpdateEvent, DeleteEvent, ListEvents, AddMember, RemoveMember, UseDonations.

```
(venv) PS C:\Users\gasto\Documents\GitHub\ong-empuje-comunitario\servers\test> python TestDonation.py
Starting DonationService Tests
=====

=== Testing DonationService ===
CreateDonation: success=True, message=Donation created successfully
ListDonations: found 14 donations
- Donation: id=1, description=Updated Donation, cantidad=200, eliminado=False
- Donation: id=2, description=Winter Clothes, cantidad=20, eliminado=False
- Donation: id=8, description=Test Donation, cantidad=100, eliminado=False
- Donation: id=9, description=Event Donation, cantidad=50, eliminado=False
- Donation: id=11, description=Event Donation, cantidad=50, eliminado=False
- Donation: id=13, description=Event Donation, cantidad=50, eliminado=False
- Donation: id=15, description=Event Donation, cantidad=50, eliminado=False
- Donation: id=16, description=Event Donation, cantidad=50, eliminado=False
- Donation: id=17, description=Event Donation, cantidad=50, eliminado=False
- Donation: id=19, description=Event Donation, cantidad=50, eliminado=False
- Donation: id=20, description=Event Donation, cantidad=50, eliminado=False
- Donation: id=21, description=Event Donation, cantidad=50, eliminado=False
- Donation: id=23, description=Event Donation, cantidad=50, eliminado=False
- Donation: id=24, description=Test Donation, cantidad=100, eliminado=False
UpdateDonation: success=True, message=Donation updated successfully
DeleteDonation: success=True, message=Donation deleted successfully

=====
DonationService Tests Completed
(venv) PS C:\Users\gasto\Documents\GitHub\ong-empuje-comunitario\servers\test>
```

Ejecución del archivo TestDonation, el cuál contiene las definiciones de los métodos para CreateDonation, UpdateDonation, DeleteDonation, ListDonations.

```
(venv) PS C:\Users\gasto\Documents\GitHub\ong-empuje-comunitario\servers\test> python TestAuthentication.py
Starting AuthService Tests
=====

=== Testing AuthService ===
Login: success=True, message=Login successful
- Logged in user: id=2, username=Janedoe
Login (invalid password): success=False, message=User not found

=====

AuthService Tests Completed
(venv) PS C:\Users\gasto\Documents\GitHub\ong-empuje-comunitario\servers\test>
```

Ejecución del archivo Test Authentication, el cuál contiene las definiciones de los métodos para evaluar las credenciales de inicio de sesión con Login.

Base de datos:

Ejecución de `SELECT * FROM roles;`

#	1 id	2 name
1	1	PRESIDENTE
2	2	VOCAL
3	3	COORDINADOR
4	4	VOLUNTARIO

Ejecución de `SELECT * FROM users;`

#	1 id	2 username	3 first_name	4 last_name	5 phone	6 password_hash	7 email	8 is_active	9 created_at	10 role_id
1	1	Leonel	Kennedy	Doe	0987654321	\$2b\$12\$YpKOidXas.8...	updated@example.com	1	2025-09-13 19:55:00	1
2	2	Janedoe	Jane	Doe	0987654321	\$2b\$12\$YpKOidXas.8...	jane.doe@example.com	1	2025-09-13 19:55:00	2
3	3	Bobsmith	Bob	Smith	5551234567	\$2b\$12\$YpKOidXas.8...	bob.smith@example.com	0	2025-09-13 19:55:00	2
4	19	Adam	Jhonson	Robinson	1112223333	\$2b\$12\$YN75fqBTuk...	eventuser@example.com	1	2025-09-15 19:54:59	1
5	21	Robert	Lauren	Jhonson	1112223333	\$2b\$12\$W9UyNuOD...	eventuser@example.com	1	2025-09-15 20:11:03	1
6	7	Sam	Robinson	Doe	1112223333	\$2b\$12\$AqETRpT...	eventuser@example.com	1	2025-09-13 22:57:48	1
7	18	Saphiro	Robinson	Smith	1112223333	\$2b\$12\$4bvx22HgR...	eventuser@example.com	1	2025-09-15 19:49:41	1
8	8	Ben	Epstein	Smith	0987654321	\$2b\$12\$0jkb.LgW9r...	updated@example.com	1	2025-09-13 23:01:31	3

Ejecución de `SELECT * FROM categories;`

#	1 id	2 name
1	1	ROPA
2	2	ALIMENTOS
3	3	UTILES_ESCOLARES
4	4	JUGUETES

Ejecución de `SELECT * FROM donations;`

#	1 id	2 description	3 quantity	4 is_deleted	5 created_at	6 created_by	7 updated_at	8 updated_by	9 category_id	10 user_id
1	1	Updated Donation	200	0	2025-09-13 19:56:08	1	2025-09-13 22:57:47	21	1	1
2	2	Winter Clothes	20	0	2025-09-13 19:56:09	2	2025-09-13 14:23:36	21	2	2
3	8	Test Donation	100	0	2025-09-13 22:57:47	2	2025-09-14 14:23:35	2	1	7
4	9	Event Donation	50	0	2025-09-13 22:57:48	3	2025-09-15 14:23:37	7	1	18
5	11	Event Donation	50	0	2025-09-13 23:01:32	1	2025-09-12 14:23:38	2	1	7
6	13	Event Donation	50	0	2025-09-13 23:06:06	2	2025-09-09 13:23:38	18	1	21
7	15	Event Donation	50	0	2025-09-13 23:50:48	21	2025-09-02 14:23:39	7	1	1
8	16	Event Donation	50	0	2025-09-15 18:42:56	2	2025-09-03 14:23:39	2	1	1
9	17	Event Donation	50	0	2025-09-15 18:53:40	21	2025-09-17 14:23:40	8	1	19
10	19	Event Donation	50	0	2025-09-15 19:49:46	1	2025-09-04 14:23:40	8	1	18
11	20	Event Donation	50	0	2025-09-15 19:55:05	3	2025-09-08 14:23:41	3	1	2
12	21	Event Donation	50	0	2025-09-15 20:11:07	21	2025-09-05 14:23:41	18	1	19

Ejecución de `SELECT * FROM events;`

#	1 id	2 name	3 description	4 event_datetime
1	1	Community Cleanup	Clean the local park	2025-10-01 10:00:00
2	2	Food Drive	Collect food donations	2025-10-15 14:00:00
3	7	Parish visit	Parish visit and do a monthly pray	2025-09-13 19:57:48
4	8	Visit orphanage	Go to send toys to the childs	2025-09-13 20:01:32
5	11	Distribution of the garment	Distribution of the garment to the poorest	2025-09-15 15:42:56

Ejecución de `SELECT * FROM event_donations;`

#	1 id	2 quantity_used	3 event_id	4 donation_id
1	1	10	1	1
2	4	1	7	9
3	5	1	8	11
4	8	1	11	16

Ejecución de `SELECT * FROM user_events;`

#	1 id	2 registration_date	3 user_id	4 event_id
1	1	2025-09-13 19:56:12	1	7
2	12	2025-09-15 19:56:12	8	1
3	13	2025-09-14 19:56:12	2	2
4	14	2025-09-16 19:56:12	21	2
5	15	2025-09-17 19:56:12	7	11

Segunda Parte

Pendientes:

integrar transferir donaciones y ofrecer donaciones con el frontend.

Estrategia de resolución

Tecnologías utilizadas:

A las tecnologías de la primera entrega se agregaron:

- **Apache Kafka:** sistema de mensajería distribuido para el intercambio de datos en tiempo real.
- **Apache Zookeeper:** servicio de coordinación y gestión del clúster Kafka.
- **Docker:** herramienta de contenedorización utilizada para desplegar y ejecutar los servicios en entornos aislados.
- **Kafka-UI:** Herramienta de visualización gráfica de mensajería implementada mediante Apache Kafka, productor y consumidor de mensajes/tópicos.

Descripción de la resolución:

Donation Request

La implementación de la tabla de la base de datos de donation request consistió en lo siguiente, se diseñó la tabla correspondiente con los siguientes atributos: request_id, organization_id, is_deleted, cada donation request posee una lista de donation request item referenciadas por claves foráneas a la tabla de la base de datos que contienen: id, request_id, organization_id, category_id, description.

La implementación de los tópicos de donation request funciona de la siguiente manera, se envía un objeto json en este formato:

```
{
  "request_id": 100,
  "organization_id": 1,
  "items": [
    {
      "category_id": 1,
      "description": "Puré de tomates"
    },
    {
      "category_id": 1,
      "description": "Arroz"
    }
  ]
}
```

Cada ítem se verá referenciado mediante la clave foránea request_id, armando correspondientemente la lista de ítems de la donation request.

Donation Cancellation Request

La implementación del donation cancellation request se encuentra definido por la tabla en la base de datos de la siguiente manera, en este caso, se reutiliza la tabla donation request: request_id, organization_id, is_deleted, se asigna is_deleted en true para marcarla como cancelada.

La implementación de los tópicos del donation cancellation request consiste en el formato json que se envía desde el consumer empleando Apache Kafka para realizar la persistencia en la base de datos, siguiendo el siguiente formato:

```
{  
  "request_id": 100,  
  "organization_id": 1  
}
```

Transferir donaciones

En la implementación se definieron las tablas donation_transfer con los siguientes atributos transfer_id, organization_id, request_id, received, processed, y la tabla donation_transfer_item con los atributos category_id, description, quantity, transfer_id.

El topic consiste en el envío de un json con el siguiente formato:

```
{  
  "request_id": 100,  
  "organization_id": 1,  
  "items": [  
    {  
      "category_id": 1,  
      "description": "Remeras",  
      "quantity": 20  
    }  
  ]  
}
```

Con la información recibida se verifica si la donación transferida es de nuestra organización hacia otra y en dicho caso se comprueba el inventario para confirmar que podemos hacer la transferencia y descontar del mismo.

Ofrecer donaciones

Para la implementación de ofrecer donaciones se crearon las tablas donation_offers que cuenta con los siguientes atributos id, available, created_at, organization_id, offer_id además de una tabla donation_offer_item con los atributos category_id, description, quantity, offer_id.

El topic consiste en el envío de un json con el siguiente formato:

```
[
  {
    "category_id": 1,
    "description": "Remeras",
    "quantity": 50
  }
]
```

Eventos

En cuanto al modelo de base de datos, se agregaron las siguientes tablas:

organization: Representa las diferentes organizaciones que se suman al sistema, pudiendo publicar/despublicar eventos y escuchar publicaciones/de publicaciones, así como también adherir miembros a eventos externos y escuchar adhesiones de miembros externos a eventos publicados.

voluntaries: Representan los voluntarios adheridos a nuestros eventos publicados.

voluntary_events: Representa la tabla intermedia que asocia voluntarios a eventos.

En la tabla **events** se agregaron los campos:

is_published, de tipo booleano para establecer si el eventos está publicado o no.

remote id, para saber si es un evento externo el id del mismo. Si es nulo es un evento de la propia organización.

organization_id, para saber a qué organización pertenece el evento.

- Estructura de mensajes
 - eventos-solidarios

```
{
  "event_id": 45,
  "organization_id": 1,
  "name": "sadas",
  "description": "sadasda",
  "datetime": "2025-10-10T21:50:00"
}
```

- baja-eventos-solidarios

```
{
  "event_id": 8,
  "organization_id": 3
}
```

- adhesión-evento

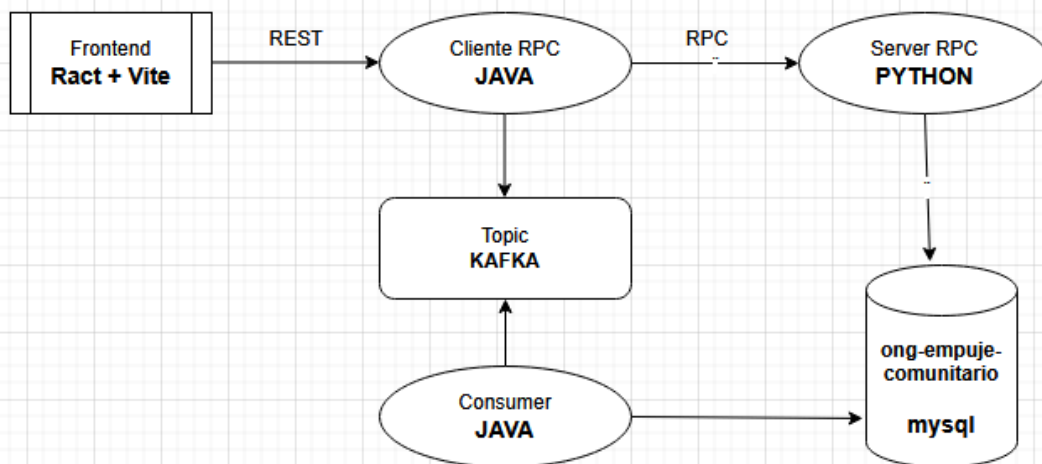
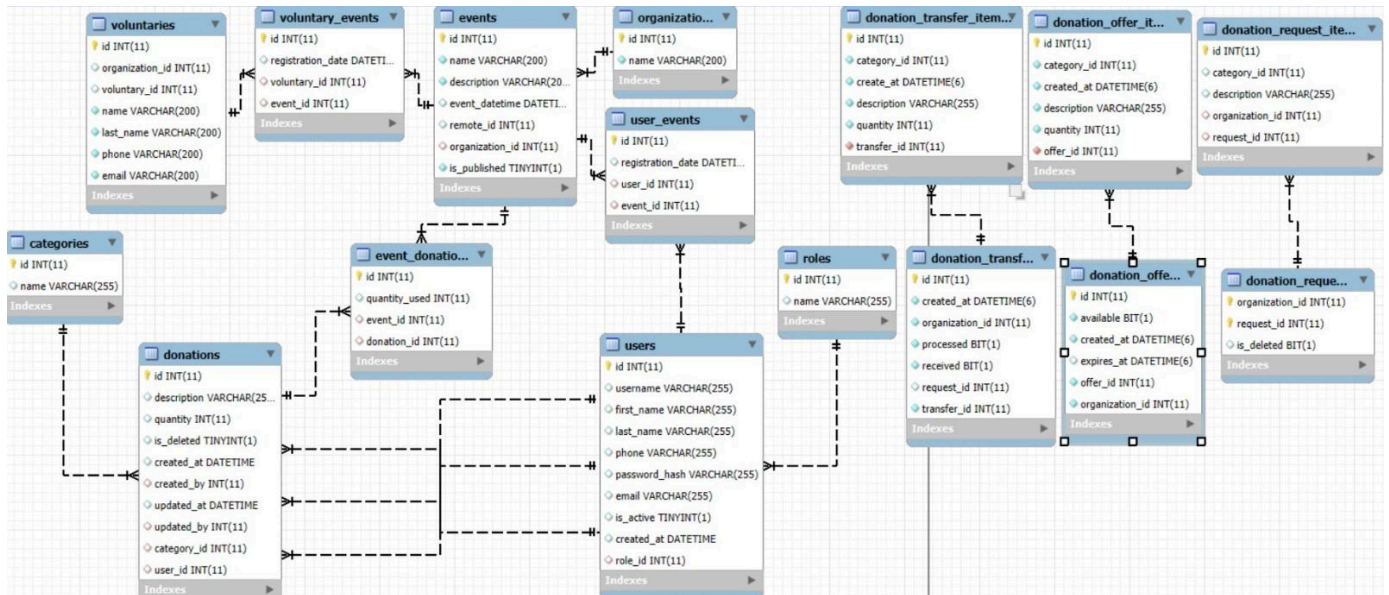
```
{
  "remote_id": 8,
  "origin_organization_id": 3,
  "voluntary": {
    "organization_id": 1,
    "voluntary_id": 3,
    "name": "Usuario",
    "last_name": "Dos",
    "phone": "1192223311",
    "email": "usuario2@hotmail.com"
  }
}
```

- Acciones frente a eventos

- eventos-solidarios: Si "organization_id" es diferente al de mi organización se persiste en la base el evento en la tabla events, con el correspondiente "remote_id" y "organization_id".
- baja-eventos-solidarios: Si el "organization_id" es diferente al de mi organización se busca en la tabla eventos por "organization_id" y "remote_id" para eliminar el evento externo.
- adhesión-evento: Si "origin_organization_id" es el de mi organización, busco en mi tabla de voluntaries por "organization_id" y "voluntary id". Si existe hago un update de los datos, sino creo el voluntario. Por último asocio el voluntario al evento, cargándose en la tabla voluntary_events.

Si es un usuario de mi organización ("organization_id" en "voluntary") agrego el usuario al evento externo, ya que es de mi organización. Lo cargo en la tabla user_events.

Diagramas:



Problemas/Errores:

Donation Request

Principalmente se han encontrado conflictos en la implementación del consumer, la comunicación utilizando Apache Kafka, frecuentaba errores de comunicación el contenedor levantado con Docker, la inicialización del puerto 9092 daba conflictos de inicio de sesión debido a Docker y problemas de comunicación debido a mal definición de las clases java en el proyecto Spring Boot de Consumer, utilizado principalmente para consumir mensajes de Apache Kafka.

Otro de los inconvenientes presentados fue el conflicto de configuración de la base de datos y de la persistencia de los datos, los conflictos de comunicación entre Apache Kafka y la base de datos MySQL, generaba conflictos al no registrar registros en la base de datos, producto de un planteamiento erróneo del tópico definido en el Consumer.

Finalmente, se han encontrado conflictos de integración con el proyecto conjunto del Cliente, el cual poseía el productor de mensajes Apache Kafka, un error en la sincronización de los mensajes generaba un conflicto al no gestionar adecuadamente los controller para redirigir el mensaje y registrarlo en la base de datos.

Donation Transfer

Conflicto en la persistencia de los datos, algunas de las transferencias realizadas no se registraban en la base de datos, se soluciono con una corrección de las entidades pertinentes.

Errores en la sincronización del consumer y producer a partir de la definición errónea de los topics.

Eventos

El principal desafío consistió en la sincronización a la hora de leer los mensajes, de manera que la consistencia del sistema no se vea afectada, y pueda reflejarse en todas las organizaciones. Por este motivo se agregó el campo "remote_organization_id" para diferenciar la organización creadora del evento, de la organización de la cual es parte el voluntario que se quiere agregar al evento ("organization_id", dentro de "voluntary").

De esta forma se puede diferenciar el origen del evento, y el origen del voluntario de forma unívoca.



Pruebas

Donation Request



juanperez
PRESIDENTE

Gestión de Usuarios

Gestión de Eventos

Donaciones

Solicitudes de Donación

Solicitudes de Donación

Crear Solicitud

ID Solicitud	Organización	Ítems	Estado	Acciones
1	ONG EMPUJE COMUNITARIO	ALIMENTOS: Papas ALIMENTOS: Arroz	Activa	Cancelar
34287	ONG EMPUJE COMUNITARIO	ALIMENTOS: Papas	Cancelada	
50879	ONG EMPUJE COMUNITARIO	ALIMENTOS: Papas	Cancelada	
84740	ONG EMPUJE COMUNITARIO	ALIMENTOS: Arroz	Cancelada	
143805	ONG EMPUJE COMUNITARIO	ALIMENTOS: Papas	Cancelada	
377288	ONG ABRIENDO CAMINOS	ALIMENTOS: Papas	Activa	Cancelar
692072	ONG EMPUJE COMUNITARIO	ALIMENTOS: Papas	Cancelada	
743707	ONG EMPUJE COMUNITARIO	ALIMENTOS: Arroz	Cancelada	
773628	ONG SOMOS MAS	ALIMENTOS: Arroz	Cancelada	
953703	ONG EMPUJE COMUNITARIO	ALIMENTOS: Arroz	Cancelada	



juanperez
PRESIDENTE

Gestión de Usuarios

Gestión de Eventos

Donaciones

Solicitudes de Donación

Solicitudes de Donación

Crear Solicitud

ID Solicitud	Organización	Ítems	Estado	Acciones
1	ONG EMPUJE COMUNITARIO	ALIMENTOS: Papas ALIMENTOS: Arroz	Activa	Cancelar
34287			Cancelada	
50879			Cancelada	
84740			Cancelada	
143805			Cancelada	
377288			Activa	Cancelar
692072			Cancelada	
743707	ONG EMPUJE COMUNITARIO	ALIMENTOS: Arroz	Cancelada	
773628	ONG SOMOS MAS	ALIMENTOS: Arroz	Cancelada	
953703	ONG EMPUJE COMUNITARIO	ALIMENTOS: Arroz	Cancelada	

Crear Solicitud

Organización

ONG EMPUJE COMUNITARIO

ALIMENTOS

Papas

Eliminar

Agregar Ítem

Cancelar

Guardar



juanperez
PRESIDENTE

Gestión de Usuarios

Gestión de Eventos

Donaciones

Solicitudes de Donación

Solicitudes de Donación

Crear Solicitud

ID Solicitud	Organización	Ítems	Estado	Acciones
1	ONG EMPUJE COMUNITARIO	ALIMENTOS: Papas ALIMENTOS: Arroz	Activa	Cancelar
34287	ONG EMPUJE COMUNITARIO	ALIMENTOS: Papas	Cancelada	
50879	ONG EMPUJE COMUNITARIO	ALIMENTOS: Papas	Cancelada	
84740	ONG EMPUJE COMUNITARIO	ALIMENTOS: Arroz	Cancelada	
143805	ONG EMPUJE COMUNITARIO	ALIMENTOS: Papas	Cancelada	
377288	ONG ABRIENDO CAMINOS	ALIMENTOS: Papas	Activa	Cancelar
692072	ONG EMPUJE COMUNITARIO	ALIMENTOS: Papas	Cancelada	
733983	ONG SOMOS MAS	ALIMENTOS: Papas	Activa	Cancelar
743707	ONG EMPUJE COMUNITARIO	ALIMENTOS: Arroz	Cancelada	
773628	ONG SOMOS MAS	ALIMENTOS: Arroz	Cancelada	
953703	ONG EMPUJE COMUNITARIO	ALIMENTOS: Arroz	Cancelada	

Donation Cancellation Request



juanperez
PRESIDENTE

Gestión de Usuarios

Gestión de Eventos

Donaciones

Solicitudes de Donación

Solicitudes de Donación

Crear Solicitud

ID Solicitud	Organización	Ítems	Estado	Acciones
1	ONG EMPUJE COMUNITARIO	ALIMENTOS: Papas ALIMENTOS: Arroz	Activa	Cancelar
34287	ONG EMPUJE COMUNITARIO	ALIMENTOS: Papas	Cancelada	
50879	ONG EMPUJE COMUNITARIO	ALIMENTOS: Papas	Cancelada	
84740	ONG EMPUJE COMUNITARIO	ALIMENTOS: Arroz	Cancelada	
143805	ONG EMPUJE COMUNITARIO	ALIMENTOS: Papas	Cancelada	
377288	ONG ABRIENDO CAMINOS	ALIMENTOS: Papas	Activa	Cancelar
692072	ONG EMPUJE COMUNITARIO	ALIMENTOS: Papas	Cancelada	
733983	ONG SOMOS MAS	ALIMENTOS: Papas	Activa	Cancelar
743707	ONG EMPUJE COMUNITARIO	ALIMENTOS: Arroz	Cancelada	
773628	ONG SOMOS MAS	ALIMENTOS: Arroz	Cancelada	
953703	ONG EMPUJE COMUNITARIO	ALIMENTOS: Arroz	Cancelada	



juanperez
PRESIDENTE

Gestión de Usuarios

Gestión de Eventos

Donaciones

Solicitudes de Donación

Solicitudes de Donación

Crear Solicitud

ID Solicitud	Organización	Items	Estado	Acciones
1	ONG EMPUJE COMUNITARIO	ALIMENTOS: Papas ALIMENTOS: Arroz	Activa	Cancelar
34287	ONG EMPUJE COMUNITARIO	ALIMENTOS: Papas	Cancelada	
50879	ONG EMPUJE COMUNITARIO	ALIMENTOS: Papas	Cancelada	
84740	ONG EMPUJE COMUNITARIO	ALIMENTOS: Arroz	Cancelada	
143805	ONG EMPUJE COMUNITARIO	ALIMENTOS: Papas	Cancelada	
377288	ONG ABRIENDO CAMINOS	ALIMENTOS: Papas	Activa	Cancelar
692072	ONG EMPUJE COMUNITARIO	ALIMENTOS: Papas	Cancelada	
733983	ONG SOMOS MAS	ALIMENTOS: Papas	Cancelada	
743707	ONG EMPUJE COMUNITARIO	ALIMENTOS: Arroz	Cancelada	
773628	ONG SOMOS MAS	ALIMENTOS: Arroz	Cancelada	
953703	ONG EMPUJE COMUNITARIO	ALIMENTOS: Arroz	Cancelada	

Donation Offer

POST



http://localhost:8081/api/donations/offer

Params

Authorization ●

Headers (10)

Body ●

Scripts

Settings

☐ none

☐ form-data

☐ x-www-form-urlencoded

☒ raw

☐ binary

```
1  {
2    "category_id": 1,
3    "description": "Remeras",
4    "quantity": 50
5  }
6
7  }
```

Body

Cookies

Headers (14)

Test Results



{ } JSON

▶ Preview

🖼 Visualize

```
1  {
2    "isSuccess": true,
3    "message": "Offer published successfully: 1412484368"
4  }
```

Key	Value	Headers
<pre>{ "offer_id": 1412484368, "organization_id": 1, "items": [{ "category_id": 1, "description": "Remeras", "quantity": 50 }] }</pre>		

Donation Transfer

POST ▼ http://localhost:8081/api/donations/transfer/1/2

Params Authorization ● Headers (10) **Body ●** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary

```
1  [
2    {
3      "category_id": 1,
4      "description": "Remeras",
5      "quantity": 15
6    }
7  ]
8
```

Body Cookies Headers (14) Test Results | ⌚

{ } JSON ▼ ▶ Preview 🖼 Visualize ▼

```
1  {
2    "isSuccess": true,
3    "message": "Transfer made to the organization: 1"
4  }
```

```
Key Value Headers
{
  "request_id": 2,
  "organization_id": 1,
  "items": [
    {
      "category_id": 1,
      "description": "Remeras",
      "quantity": 15
    }
  ]
}
```

Eventos

Eventos-solidarios

- Desde el front tenemos un botón que dispara el envío del mensaje al tópico.

evento	evento	12/10/2025, 02:11:00	Ver	Agregarse	Modificar	Eliminar	Publicar
--------	--------	----------------------	-----	-----------	-----------	----------	----------

- Una vez publicado lo vemos de la siguiente manera.

evento	evento	12/10/2025, 02:11:00	Ver	Agregarse	Modificar	Eliminar	✓ Publicado
--------	--------	----------------------	-----	-----------	-----------	----------	-------------

- Con la herramienta gráfica podemos ver el mensaje enviado al tópico.

Key	Value	Headers
-----	-------	---------

```
"event_id": 48,  
"organization_id": 1,  
"name": "evento",  
"description": "evento",  
"datetime": "2025-10-12T14:11:00"
```

- Si envíamos el evento simulando que viene de otra organización

Produce Message

Close

Partition

Partition #0

Key Serde

String

Value Serde

String

☐ Keep contents

Key

1

Value

1

```
"event_id": 1,
"organization_id": 3,
"name": "evento externo ",
"description": "evento externo",
"datetime": "2025-10-12T14:11:00"
```

7

- Desde el front, en la sección de eventos externos vemos

Eventos Externos

Nombre	Descripción	Fecha	Organizador	Acciones
evento externo	evento externo	12/10/2025, 02:11:00	ONG CREANDO LAZOS	Agregarse



Baja-event-solidario

- Si le damos a eliminar enviaremos el mensaje al tópico para que las demás organizaciones actualicen la lista de sus eventos.

evento	evento	12/10/2025, 02:11:00	Ver	Agregar	Modificar	Eliminar	✔ Publicado
--------	--------	----------------------	---------------------	-------------------------	---------------------------	--------------------------	-----------------------------

Adhesión-evento

- Si nos adherimos al evento externo, con el botón agregarse, se enviará al tópico el mensaje con la información correspondiente

Eventos Externos

Nombre	Descripción	Fecha	Organizador	Acciones
evento externo	evento externo	12/10/2025, 02:11:00	ONG CREANDO LAZOS	Agregar

- Desde el tópico vemos lo siguiente. Los datos del voluntario, de la organización a la que pertenece, y el de la organización origen del evento.

Key	Value	Headers
<pre>{ "remote_id": 1, "origin_organization_id": 3, "voluntary": { "organization_id": 1, "voluntary_id": 2, "name": "Federico", "last_name": "Acosta", "phone": "1162519312", "email": "fedecosta07@hotmail.com" } }</pre>		

- Si recibimos la adhesión de un voluntario a un evento publicado por nuestra organización.

Key Value Headers

```
{
  "remote_id": 48,
  "origin_organization_id": 1,
  "voluntary": {
    "organization_id": 3,
    "voluntary_id": 2,
    "name": "Juan",
    "last_name": "Lopez",
    "phone": "1162519312",
    "email": "juan_lopez@hotmail.com"
  }
}
```

- Luego en el front si vemos los miembros del evento, veremos agregado el voluntario al mismo.

Eventos Solidarios Agregar Evento

Nombre	Descripción	Fecha	Miembros	Acciones
evento	evento	12/10/2025, 02:11:00	Ver	Agregar Modificar Eliminar Publicado
sadas	sadasda	12/10/2025, 02:50:00	Ver	Agregar Modificar Eliminar Publicado
ggg	sadadasd			Agregar Modificar Eliminar Publicado
bbbb	lllll			Agregar Modificar Eliminar Publicado
ssss	ssssss			
gffsada	dasdasdasda	4/10/2025, 10:41:00	Ver	

Miembros de evento

juan_lopez@hotmail.com

Eliminar

Buscar usuario...

Agregar

Tercera parte

Descripción de la resolución:

Informe de donaciones (GRAPHQL):

Se ha desarrollado una vista en el frontend para visualizar los reportes con sus respectivos filtros habilitados, se habilitó una sección de creación, modificación y eliminación de filtros personales del usuario logueado.

Se ha desarrollado un servidor establecido como microservicio en un puerto específico en el 10050, el cual presta el servicio de Web Services, incluyendo las instrucciones y consultas GraphQL del proyecto en cuestión.

Se ha empleado el desarrollo de clases .java para los Controller, los Services, los Repository y los Models para realizar las peticiones directamente con la base de datos mediante GraphQL.

DonationReportController se encarga de gestionar las peticiones URL del frontend para ejecutar las consultas GraphQL correspondientes en relación a las donaciones disponibles.

Filtro personalizado del informe de donaciones (GRAPHQL):

Se ha desarrollado un conjunto de filtros personalizables por el usuario, en donde en la base de datos se refleja el usuario con sus respectivos filtros. Se puede filtrar por categoría, por activo o inactivo y por fechas.

Principalmente se han diseñado nuevas tablas en la base de datos para generar la comunicación entre el backend y el frontend, de esta forma se consultan los filtros guardados del usuario autenticado en la sesión actual. Modificando el resto de filtros correspondientemente a los del usuario.

Informe Excel de donaciones (REST):

Para el reporte fueron necesarias las clases category y donation con sus respectivos Repository

para obtener la información de la base de datos. En el pom se agregó las dependencias de apache.poi para la generación del excel.

Se creó un controller que se encarga de recibir la solicitud para generar el excel y delega esta tarea

al servicio.

La clase service del reporte excel consta de un método principal en el cual se guardan los datos

de las donaciones y se genera un archivo excel que contiene una hoja de resumen general,

una hoja por categoría de donación y una hoja vacía si no hay resultados.

Las solicitudes tienen el siguiente formato:

```
{
  "categoryId": 0,
  "startDate": "2025-10-25T18:12:37.468Z",
  "endDate": "2025-10-25T18:12:37.468Z",
  "deleted": true
}
```

y devuelve como resultado un file Excel descargable.

Consulta de Presidentes y ONGs (SOAP):

El service contiene 2 métodos principales los cuales se encargan de crear las Request que se hacen al WSDL “<https://soap-app-latest.onrender.com/?wsdl>” una para consultar las ONGs y otra para los Presidentes de dichas ONGs, luego guarda las respuesta en una variable para hacer un parseo y obtener los datos que necesitamos id, nombre, dirección, etc.

Las solicitudes se hacen en formato de XML pero la única información que debemos enviar son los IDs de las ONGs que queremos consultar.

```
[
  6,7,8,9|
]
```

Problemas/Errores:

Consulta de Presidentes y ONGs (SOAP): Errores en la creación del XML para hacer la Request al WSDL y en el parseo por problemas con el namespace que hacía que en la respuesta no me devolviera los datos de las ONGs y Presidentes. También existieron problemas de compatibilidad relacionados con las dependencias de OXM por lo que se optó por el uso de RestTemplate en vez de WebServiceTemplate.

Informe de donaciones (GRAPHQL):

Principalmente problemas y conflictos con la definición de las consultas GraphQL, la comunicación entre el frontend y el backend, debido a una mal instrucción en la Mutation o en la Query del schema.graphqls.

Filtro personalizado del informe de donaciones (GRAPHQL):

Errores de comunicación con la base de datos, errores en la clave foránea, clave primaria, datos vacíos y consultas a columnas incorrectas en la tabla SavedFilter.

Pruebas:

Link del vídeo de la defensa al Drive:

<https://drive.google.com/file/d/1sm38IHqfF9nBYzsWDo9IlujqljFHQhJ1/view?usp=sharing>

Link del vídeo de la defensa a Youtube:

<https://www.youtube.com/watch?v=DjShdrbFEA4>